

科目：データベース演習

第11回：SQL応用：様々なジョイン

講師：鈴木源吾

前回の復習

1. 関数で新しいカラムを作り出す
 - 関数の使い方
 - 数値演算・文字列演算
2. ジョインの基礎
 - 正規化とジョイン
 - ジョインでテーブルを連結する

今回のトピック

1. 外部ジョインで欠落のあるデータを扱う
2. 一歩進んだジョイン
3. 組合せを生成するジョインでバスケット分析

その他の資料：第7章

topic 1

外部ジョインで欠落のあるデータを扱う

ジョインの振り返り（内部ジョイン）

- ジョインでは、2つのテーブルに対して、条件を満たす（例：customer_idが等しい）行があるときに、その2つの行を結びつけて、1つの行にして返却する処理を行う
- 条件を満たさない場合は、演算の結果には入らない
- 後述する外部ジョインとの対比で、内部ジョイン（inner join）とも呼ばれる

access_log テーブル

request_time	request_method	request_path	customer_id
2014-12-29 03:34:01	GET	/	54115
2014-12-29 03:34:05	GET	/search	104097
2014-12-29 03:34:34	GET	/	324
2014-12-29 03:34:40	GET	/	104097

customers テーブル

customer_id	customer_name
324	葛飾道夫
54115	山田三郎
104097	丸山冴子



ジョインにおける対応する行（ジョインの相手）

- このジョインの例における，対応する行を考えてみよう（ジョインの相手）
- access_logテーブルの1番上の行で，アクセスした顧客のcustomer_idは54115
- そのジョインの相手は，同じ顧客のcustomer_idを持つ，山田三郎さんの行である

access_log テーブル

request_time	request_method	request_path	customer_id
2014-12-29 03:34:01	GET	/	54115
2014-12-29 03:34:05	GET	/search	104097
2014-12-29 03:34:34	GET	/	324
2014-12-29 03:34:40	GET	/	104097

customers テーブル

customer_id	customer_name
324	葛飾道夫
54115	山田三郎
104097	丸山冴子

access_logとcustomersの両方にある
customer_idがジョインの相手となる

1000
2000
3000
...

ジョインにおける対応する行（ジョインの相手）

- access_logテーブルのcustomer_idはnullをとるときもある
 - 顧客登録していない利用者によるアクセスなどが考えられる
- このようなログの情報はジョインの相手がいないので、ジョインの結果で失われてしまう

access_log テーブル

request_time	request_method	request_path	customer_id
2014-12-29 03:34:01	GET	/	54115
2014-12-29 03:34:05	GET	/search	104097
2014-12-29 03:34:34	GET	/	324
2014-12-29 03:34:40	GET	/	104097

customers テーブル

customer_id	customer_name
324	葛飾道夫
54115	山田三郎
104097	丸山牙子

ログインしていない場合customer_idがnullとなる→対応する顧客がない

1000
2000
...
null

ジョインの相手がいない

ジョインにおける対応する行（ジョインの相手）

- 今度は逆にcustomersテーブル側から見てみよう
- もし、顧客の中に全然アクセスしていない人がいたとすると、customersテーブルにデータはあるが、access_logテーブルに対応するデータはないことがある
- この場合は、ジョインするとこの顧客（3000番）の情報は失われる

access_log テーブル

request_time	request_method	request_path	customer_id
2014-12-29 03:34:01	GET	/	54115
2014-12-29 03:34:05	GET	/search	104097
2014-12-29 03:34:34	GET	/	324
2014-12-29 03:34:40	GET	/	104097

customers テーブル

customer_id	customer_name
324	葛飾道夫
54115	山田三郎
104097	丸山冴子

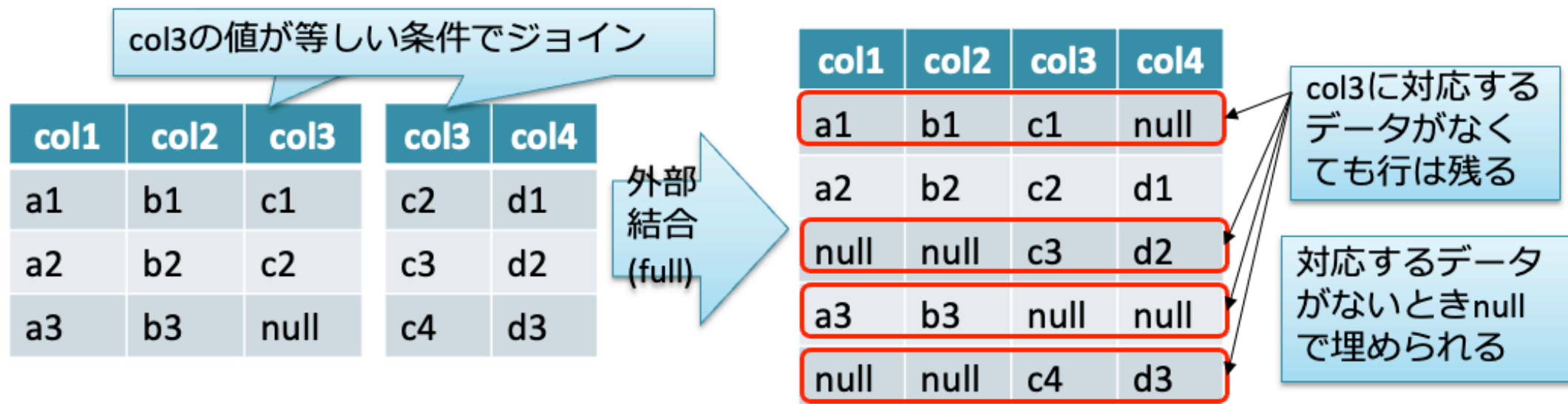
access_logとcustomersの両方にあるcustomer_idがジョインの相手となる

1000
1000
2000
2000
...
...
3000
...

逆にcustomersの中に全然アクセスしない人がいると、access_logテーブルにはジョインの相手がいないことがある

対応しない行も残したい→外部結合

- しかし，顧客としてログインしていないアクセスも合わせて分析したいときもある
- 顧客の情報を分析するときに，アクセスしていない事実も必要なことがある
- そのような目的のため，対応する行がなくても行を残す，外部結合（outer join）という機能がある．イメージを下図に示す

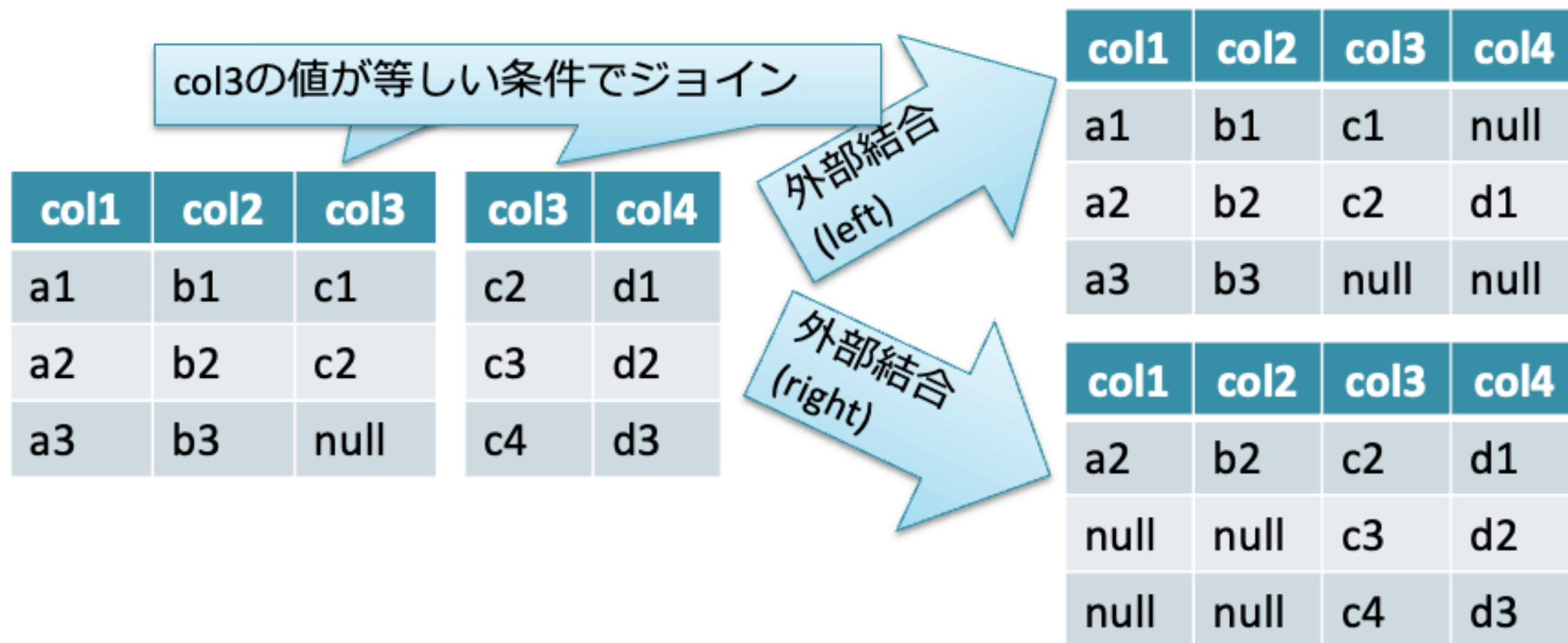


外部結合の種類

- 外部結合には, left (左側), right (右側), full (完全) の3種類がある
 - 左側外部結合, 右側外部結合, 完全外部結合と呼ぶ
- これは, どちら側のテーブルの行を保存するかを意味している
 - left→左側 (SELECT文で先に書いてある方) の行を保存
 - right→右側 (SELECT文で後に書いてある方) の行を保存
 - full→両側の行を保存
- leftを一番よく使う
 - rightか, leftかは順番だけの問題
 - 主として分析したいテーブルがあることが多く, そちらの行を保存するため

外部結合の種類

- 左側・右側外部結合の結果は下図のようになる（両側は既出）



外部結合のSQL文

前のページであげた例のSQL文は以下の通り（前ページ図の左の表をtable1とする）.

完全外部結合

```
SELECT table1.col1, table1.col2, table1.col3, table2.col4 FROM table1  
FULL OUTER JOIN table2 ON table1.col3 = table2.col3
```

左側外部結合

```
SELECT table1.col1, table1.col2, table1.col3, table2.col4 FROM table1  
LEFT OUTER JOIN table2 ON table1.col3 = table2.col3
```

右側外部結合

```
SELECT table1.col1, table1.col2, table2.col3, table2.col4 FROM table1  
RIGHT OUTER JOIN table2 ON table1.col3 = table2.col3
```

外部結合の例

例に出したaccess_logテーブルとcustomersテーブルを外部結合してみよう

customer_idが等しいという条件で、左側外部結合を行い、access_logのrequest_timeとcustomersテーブルのcustomer_nameの組を求めてみよう
(最大100個の出力制限をつけること)

- これは、customer_idがnullであっても、ジョインした結果を残すことになる

access_log テーブル

request_time	request_method	request_path	customer_id
2014-12-29 03:34:01	GET	/	54115
2014-12-29 03:34:05	GET	/search	104097
2014-12-29 03:34:34	GET	/	324
2014-12-29 03:34:40	GET	/	104097

customers テーブル

customer_id	customer_name
324	葛飾道夫
54115	山田三郎
104097	丸山冴子



外部結合のSQL文

以下のSQL文で求められる。テーブル名の別名をAS句を使って定義すると、記述量を減らすことができる（例：access_logテーブルに別名aを定義し、SELECT句・ON句ではこの別名を用いている）。その下に結果の一部を示す

```
SELECT a.request_time, c.customer_name FROM access_log AS a
LEFT OUTER JOIN customers AS c
ON a.customer_id = c.customer_id
LIMIT 100;
```

	request_time timestamp without time zone 🔒	customer_name character varying (32) 🔒
26	2012-01-01 00:22:39	岩本 さとみ
27	2012-01-01 00:23:43	岩本 さとみ
28	2012-01-01 00:25:16	岩本 さとみ
29	2012-01-01 00:21:32	[null]
30	2012-01-01 00:23:13	[null]

左側外部結合でJITエラーがでる場合の対処

環境によっては、左側外部結合を行う場合に、以下のようなエラーがでることがある

```
ERROR:  could not load library "/Library/PostgreSQL/14/lib/postgresql/llvmjit.so"
```

この場合、まず以下の行をpgAdminで実行し、JIT（実行時コンパイル）を無効化する

```
ALTER SYSTEM SET jit=off;
```

その後に設定を再読み込みする

```
select pg_reload_conf();
```

無効化されたかは、以下を実行しoffが出力されるかで確認できる

```
show jit;
```

外部結合した後のデータ数

外部結合した後のデータ数と内部結合の後のデータ数を比較してみよう。左側外部結合では、左側のテーブルのすべてのデータ数が結果に含まれることがわかる。

```
SELECT count(*) FROM access_log AS a  
LEFT OUTER JOIN customers AS c  
ON a.customer_id = c.customer_id;
```

→ 5539909 (これは、SELECT count(*) FROM access_logと同じ)

```
SELECT count(*) FROM access_log AS a  
JOIN customers AS c  
ON a.customer_id = c.customer_id;
```

→ 3729574 (access_logのcustomer_idにnullがあった行は結合されない)

例題1

customer_locationsは、顧客の居住地域（都道府県）を表すテーブルである（データは3顧客のみ）。アクセスの要求時間とアクセスした人の居住地域の対応関係を知るために、access_logテーブルにcustomer_locationsテーブルをcustomer_idが等しいことによって左側外部結合し、request_timeとprefectureの組を求めよ（顧客登録していないアクセスした人のログ（= customer_idがnull）を除外しないために外部結合を使う）。ただし、結果のデータ数を制限するために以下の指定を行うこと

- request_pathが/searchに等しいものに限定すること
- 結果は、prefectureによってソートすること（PostgreSQL→昇順, SQLite→降順とせよ。NULLが最初か最後か違うため）
- 最大100個の出力制限を付けること

アクセスした人の居住地域別にアクセス数を集計する

access_logテーブルとcustomer_locationsテーブルを使って、アクセスした人の居住地域別にアクセス数を集計してみよう。イメージを下図に示す

- prefecture毎に集計→GROUP BY句の利用
- customer_locationsテーブルには、都道府県情報が登録されていない顧客がいる。その場合についても「不明」の位置づけで集計はしたい→外部結合の利用

access_logテーブル

request_time	request_method	request_path	customer_id
2014-12-29 03:34:01	GET	/index.html	54115
2014-12-29 03:34:05	GET	/search	104097
2014-12-29 03:34:34	GET	/index.html	324
2014-12-29 03:34:40	GET	/index.html	104097
2014-12-29 03:35:05	GET	/items/531415	93292

沖縄県のアクセスは1回

customer_locationテーブル

customer_id	prefecture
54115	沖縄県
93292	島根県
104097	東京都

東京都のアクセスは2回

アクセスした人の居住地域別にアクセス数を集計する

以下のSQL文によって実施できる→時間かかるので待つこと

- access_logにcustomer_locationsを左側外部結合
- prefectureの値でグループ化する
- グループ毎に行の数を数える (count(*))

```
SELECT l.prefecture, count(*)  
FROM access_log AS a  
LEFT OUTER JOIN customer_locations AS l  
ON a.customer_id = l.customer_id  
GROUP BY l.prefecture;
```

アクセスした人の居住地域別にアクセス数を集計する

以下の実行結果が得られる

	prefecture text 	count bigint 
1	[null]	5539794
2	島根県	44
3	沖縄県	71

さらに日付別に集計する

日付別に居住地域別のアクセス数を集計してみよう。求めたいのは以下の結果である

- 集計するためには、日付のカラムが必要になる
- 以下の結果は、データの途中を切り出している（nullが多いため）

	 access_date date	 prefecture text	 count bigint
35	2012-02-04	[null]	1686
36	2012-02-05	沖縄県	6
37	2012-02-05	[null]	1534
38	2012-02-06	[null]	1482
39	2012-02-07	[null]	1802
40	2012-02-08	[null]	1387
41	2012-02-09	[null]	1803

さらに日付別に集計する

access_logテーブルには、アクセスした年時分秒の情報(request_time)はあるが、日付はない

→ これは型のキャストを使って解決できる

```
CAST(request_time AS date)
```

とdate型に変換することによって、日付を求めることができる。以下のSQL文を実行すれば、うまくいくことが確認できる

```
SELECT CAST(request_time AS date) FROM access_log LIMIT 10;
```

さらに日付別に集計する

以下のSQL文によって、日付と居住地域別のアクセス数を求めることができる

- 日付をGROUP BY句で参照するため、別名access_dateをつけた

```
SELECT CAST(request_time AS date) AS access_date, l.prefecture, count(*)  
FROM access_log AS a  
LEFT OUTER JOIN customer_locations AS l  
ON a.customer_id = l.customer_id  
GROUP BY access_date, l.prefecture;
```

演習 課題5-3

access_logテーブルとcustomer_locationsテーブルを用いて，要求されたURL

(request_path) と居住地域別のアクセス数の集計を求めたい．また，例題と同様に，居住地域が登録されていない場合の集計結果も含めたい（結果例を下図に示す）．例題を参考に，この集計を行うSQL文を作成せよ．ただし，検索結果は10個のみに制限すること

	request_path text	prefecture text	count bigint
1	/	島根県	13
2	/	沖縄県	9
3	/	[null]	1101427
4	/cart/checkout	島根県	1
5	/cart/checkout	沖縄県	2
6	/cart/checkout	[null]	32111
7	/cart/put/1	[null]	74
8	/cart/put/10	[null]	68
9	/cart/put/100	[null]	60
10	/cart/put/101	[null]	50

topic 2

一歩進んだジョイン

セルフジョイン

1つのテーブル内に含まれる情報を組み合わせたいときがある

- 同じテーブル内のカラムを，ある条件でさらに追加する
- 同じテーブル内の異なる行の値を演算する

→ 1つのテーブルを**自分自身とジョイン**することで実現でき，**セルフジョイン**と呼ぶ
(下図は，組織のテーブルから組織名を階層的に表示する例)



SQL文 : SELECT t1.組織id, t1.組織名, t2.組織名 FROM 組織 AS t1 JOIN 組織 AS t2 ON t1.組織id = t2.親組織id;

1年前との売上比較

セルフジョインは，過去との比較をするときにも使われる。
サンプルDBの年間売上を保存している， yearly_salesテーブルを用いてみよう。
求めたい結果が右側である． 1年前の結果を横に並べている

	year integer	shop_id integer	sales_amount bigint
1	2012	1	72950157
2	2012	2	140909834
3	2012	3	91359835
4	2013	1	91350983
5	2013	2	139480193
6	2013	3	92029355

	year integer	shop_id integer	curr_sales bigint	prev_sales bigint
1	2013	1	91350983	72950157
2	2013	2	139480193	140909834
3	2013	3	92029355	91359835

1年前との売上比較

セルフジョインするテーブルに対して, curr (当年→2013年), prev (前年→2012年) と別名をつけて区別する

- 結合条件は, yearとshop_idについて設定する必要がある
- shop_idは等しいという条件でOK
- yearについては, prevのyearはcurrのyearから1引いた値である必要がある

		結合条件				
		curr		prev		
	year integer	shop_id integer	sales_amount bigint		year integer	shop_id integer
1	2012	1	72950157	1	2012	1
2	2012	2	140909834	2	2012	2
3	2012	3	91359835	3	2012	3
4	2013	1	91350983	4	2013	1
5	2013	2	139480193	5	2013	2
6	2013	3	92029355	6	2013	3

1年前との売上比較

よって、求めるSQL文は以下となる。実行してみよう

```
SELECT curr.year, curr.shop_id,  
       curr.sales_amount AS curr_sales, prev.sales_amount AS prev_sales  
FROM yearly_sales AS curr  
JOIN yearly_sales AS prev  
ON curr.year -1 = prev.year AND curr.shop_id = prev.shop_id;
```

例題2

itemsテーブルは、商品の番号 (item_id)、商品の名前 (item_name)、商品の価格 (item_price)、商品を販売する店舗の番号 (shop_id) のカラムを持つ。このitemsテーブルをセルフジョインすることにより、商品のすべての組み合わせを作ることができる。さらに、**同じ店舗で購入できる**商品の組み合わせ (商品1・商品2と呼ぶ) を求めたい。そこで、店舗の番号、商品1の名前 (item_name1)、商品2の名前 (item_name2)、2つの商品の合計価格 (total_price) を求めるSQL文を作成せよ。次ページに結果の例を示す

注意

- 組合せには同じ商品の重複と、順序入れ替えの重複は許すこととする
(練習として重複を排除する方法を考えてみよう→次節に関連)

例題2 検索結果の例（一部）

	shop_id integer 	item_name1 text 	item_name2 text 	total_price integer 
1	28	Prime カットソー	Prime デニムシャツ	8030
2	28	Prime カットソー	Z3 ジャケット	18900
3	28	Prime カットソー	Z3 ポロシャツ	8750
4	28	Prime カットソー	Laser ジャケット	33210
5	28	Prime カットソー	Z3 バッグ	7910
6	28	Prime カットソー	First スニーカー	7550
7	28	Prime カットソー	AAU ニットTシャツ	11350
8	28	Prime カットソー	Prime Tシャツ	5190
9	28	Prime カットソー	Laser ポロシャツ	10410

演習 課題5-4

前に述べた「1年前との売上比較」において、前年から当年への売上額の増減額も一緒にみたいという要望があった。増減額を計算した結果も併せて表示するSQL文を作成せよ。このカラムの別名はsales_diffとせよ。以下に結果の例を示す

	year integer	shop_id integer	curr_sales bigint	prev_sales bigint	sales_diff bigint
1	2013	1	91350983	72950157	18400826
2	2013	2	139480193	140909834	-1429641
3	2013	3	92029355	91359835	669520

topic 3

組合せを生成するジョインでバスケット分析

バスケット分析

- ECサイトで買い物をすると、おすすめが表示されることがある
- どのようにおすすめを決めればよいだろう？

→ 一緒に買われやすい商品を出してあげる

- 過去の注文を分析し、傾向を調べる
 - 「このセーターとシャツは一緒に買われやすい」
 - 「プログラミングの本を買う人は、このマンガも買うようだ」

バスケット分析と呼ばれる

- 一緒にスーパーのカゴ（バスケット）に入れる商品を見つける

一緒に買われる商品を求めよう

order_detailsテーブルを利用する（下表）。ここには、注文毎に、注文番号(order_id)・注文した商品のID(item_id)・注文した商品の個数(item_qty)・注文した商品の価格(item_price)が保存されている

- 例えば、2番の注文では、325番の商品と、159番の商品が同時に購入されている

	 order_id integer	 item_id integer	 item_qty integer	 item_price integer
1	1	122	1	3600
2	2	325	1	8440
3	2	159	1	8110
4	3	364	1	7640
5	4	354	1	3370
6	5	360	1	7420
7	6	244	1	8270
8	7	235	1	18010
9	8	257	1	2100
10	9	46	2	13140

一緒に買われる商品を求めよう

- order_detailsテーブルをセルフジョインし、一緒に買われる商品を横に並べたい。
。以下の図のようなイメージである

	order_id integer	item_id integer	item_qty integer	item_price integer
1	1	122	1	3600
2	2	325	1	8440
3	2	159	1	8110
4	3	364	1	7640
5	4	354	1	3370
6	5	360	1	7420

セルフ
ジョイン

ほしい結果

order_id	item_id1	item_id2
2	325	159

一緒に買われる商品を求めよう

まずは、セルフジョインをしてみよう。以下のSQL文を実行してみよう

- 結果が大きくなりすぎるので、order_idを30000未満に限定した

```
SELECT l.order_id, l.item_id AS item_id1, r.item_id AS item_id2
FROM order_details AS l
JOIN order_details AS r
ON l.order_id = r.order_id
WHERE l.order_id < 30000 AND r.order_id < 30000;
```

一緒に買われる商品を求めよう

以下の結果が得られる。この結果には少し問題がある

1. item_id1とitem_id2が等しい場合も結果に入っている→不要
2. 注文番号2に着目すると、325と159と159と325の両方がある→片方でよい

→ 条件として、 $l.item_id < r.item_id$ を追加すれば2つの問題は解決する

	 order_id integer	 item_id1 integer	 item_id2 integer
1	1	122	122
2	2	325	159
3	2	325	325
4	3	364	364
5	4	354	354
6	2	159	159

一緒に買われる商品を求めよう

- 前ページの条件 ($l.item_id < r.item_id$) を追加し, さらに商品の組合せに対して注文がどの程度あるかを見るために, 2つの商品IDでソートしてみよう
- 注意: WHERE句に追加する条件を, $item_id1 < item_id2$ とするとエラーとなる. これは句の評価の順番に起因する (WHERE句よりSELECT句が後に評価)

```
SELECT l.order_id, l.item_id AS item_id1, r.item_id AS item_id2
FROM order_details AS l
JOIN order_details AS r
ON l.order_id = r.order_id
WHERE l.order_id < 30000 AND r.order_id < 30000
AND l.item_id < r.item_id
ORDER BY item_id1, item_id2;
```

一緒に買われる商品を求めよう

前ページのSQL文の実行結果（一部）は以下となる

- 商品4-6の組合せを含んだ注文が2回あったことがわかる

	order_id integer	item_id1 integer	item_id2 integer
1	5580	1	281
2	16668	2	110
3	1237	2	196
4	28114	2	293
5	5923	2	382
6	6249	3	50
7	7073	3	387
8	3079	4	6
9	9373	4	6
10	9373	4	287

一緒に買われる商品の注文を集計しよう

最後に、商品の組合せ毎に注文は何件あったかを求めてみよう

- これまでのSQL文を参考にして、item_id1とitem_id2でグループ化すればよい
- そして、行の数をカウントすればよい

```
SELECT l.item_id AS item_id1, r.item_id AS item_id2 , count(*)  
FROM order_details AS l  
JOIN order_details AS r  
ON l.order_id = r.order_id  
WHERE l.order_id < 30000 AND r.order_id < 30000  
AND l.item_id < r.item_id  
GROUP BY item_id1, item_id2;
```

一緒に買われる商品の注文を集計しよう

前ページのSQL文を実行した結果は以下となる（一部）

	 item_id1 integer 	item_id2 integer 	count bigint 
1	93	34	1
2	282	258	2
3	143	96	1
4	293	196	1
5	104	8	1
6	145	63	1
7	233	196	1
8	325	169	1
9	373	239	1
10	254	191	1

演習 課題5-5

前ページのSQL文の結果に対して、注文の多かった商品の組合せを知りたいという要望があった。そこで、商品の組合せに対する注文数が多い順から10行出力するようなSQL文を作成せよ。結果例を以下に示す

	 item_id1 integer	 item_id2 integer	 count bigint
1	192	75	4
2	360	259	4
3	274	257	4
4	206	118	4
5	349	310	3
6	185	19	3
7	284	217	3
8	302	86	3
9	378	162	3
10	382	151	3